ТЕМА 3. КАСКАДНЫЕ ТАБЛИЦЫ СТИЛЕЙ

ЛЕКЦИЯ 3.41. ПСЕВДОКЛАССЫ И ПСЕВДОЭЛЕМЕНТЫ.

Псевдоклассы. Псевдоэлементы.

Псевдоклассы

С самых первых занятий по css мы знаем, как обратится к элементу страницы. Можно выбрать определенный тег, добавить к нему класс или идентификатор, или тип элемента, или вообще все вместе.

Но если задуматься, то все перечисленные характеристики тега скорее статичные и относятся к положению элемента в DOM-дереве. А ведь на элементы страницы оказывают влияние еще и некоторые внешние факторы. Например, взаимодействуя с элементами, пользователь может выбрать пункт в меню, заполнить поле формы, навести курсор мыши на элемент, посетить ссылку и многое другое.

Так вот, **псевдоклассы** позволят нам управлять стилевым оформлением элемента, основываясь на таких «динамичных» параметрах (состояниях элемента). Как и с обычными классами, можно совмещать вместе в одном селекторе любое число псевдоклассов.

Общий синтаксис для всех псевдоклассов:

```
селектор:псевдокласс{свойство: значение;}
```

Доступных вариантов достаточно много, рассмотрим некоторые их них далее.

LHVA модель

Ссылки достаточно интересный элемент разметки. При внешней простоте, они ухитряются содержать в себе сразу

©Pechora-PRO

несколько различных состояний. У нас есть ссылки в состоянии по умолчанию, при наведении, при нажатии, уже посещенные или вообще недоступные. Стилизуя все эти состояния с помощью псевдоклассов важно учитывать определенную модель.

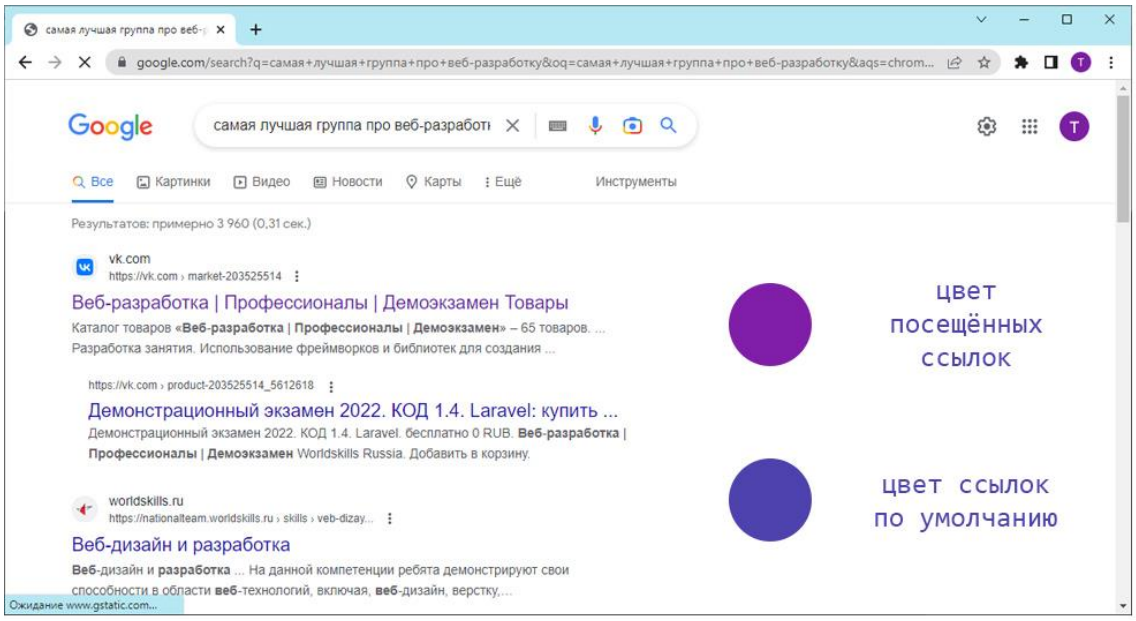
Называется она LVHA – по первым буквам псевдоклассов и определяет порядок применения стилей к элементу.

:link

CSS псевдокласс **:link** позволяет вам выбирать ссылки внутри элементов. Он выберет любую ссылку, которая ещё не была посещена, даже те, которые уже стилизованы с помощью других псевдоклассов.

:visited

В противоположность прошлому псевдоклассу **:visited** позволяет вам выбирать ссылки, которые **были посещены**. Этот стиль может переопределяться другими относящимися к ссылкам псевдоклассами, такими как **:link**, **:hover** и **:active**, появляющимися в соответствующем порядке.



Вы можете стилизовать посещенные ссылки, но существуют ограничения на то, какие стили вы можете использовать:

©Pechora-PRO

- color
- background-color
- border-color(и его подсвойства)
- column-rule-color
- outline-color
- text-decoration-color
- text-emphasis-color
- цветовые части атрибутов fill и stroke

Кроме того, даже для вышеуказанных стилей вы не сможете изменить прозрачность между непосещенными и посещенными ссылками (причем не работают ни альфа-каналы в описании цвета, ни свойство transparent).

Интересная историческая справка

Такое ограничение связано с тем, что этот селектор позволял сайтам раскрывать историю просмотров пользователя. Программы могли не только определить, где пользователь был в сети, но и угадать большое количество информации о личности пользователя.

Чтобы сгладить эту проблему, браузеры ограничили объем информации, которую можно получить по посещенным ссылкам. Теперь браузеры будут искажать информацию о посещенных страницах. Например,

- Методы и функции для проверки, всегда будут возвращать значения, указывающие на то, что пользователь никогда не посещал ни одну из ссылок на странице.
- Если вы используете родственный селектор, такой как :visited + span, соседний элемент (span в этом примере) будет оформлен так, как если бы ссылка не была посещена.

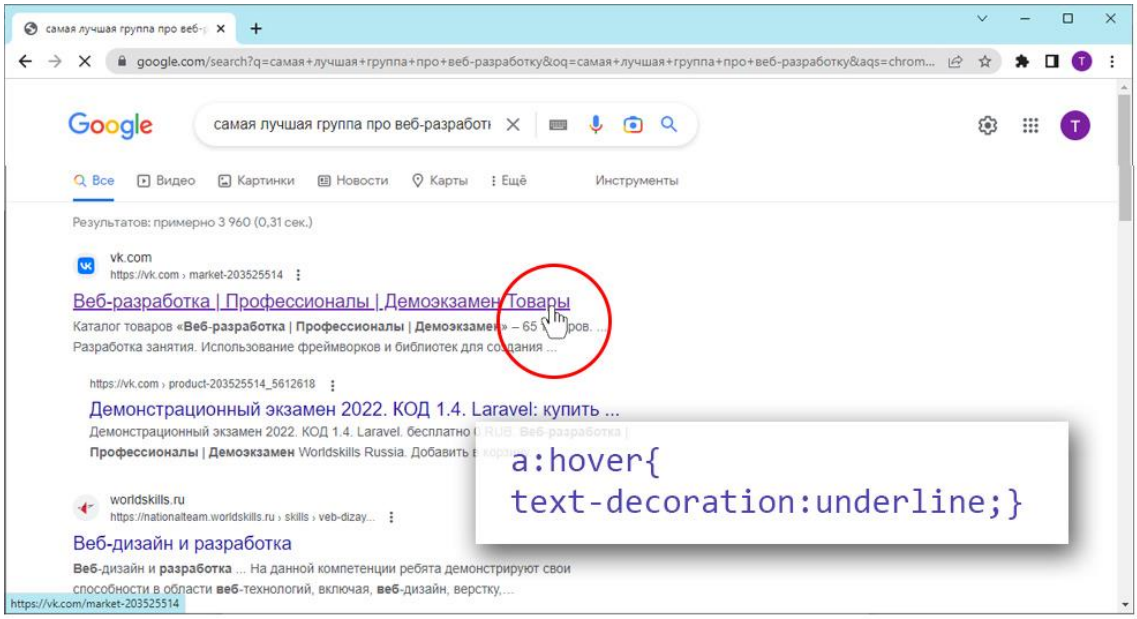
©Pechora-PRO

- В редких случаях, если вы используете вложенные элементы ссылок, и совпадающий элемент отличается от ссылки, присутствие которой в истории проверяется, элемент также будет отображаться так, как если бы ссылка не была посещена.

:hover

Наверно, самый популярный по частоте использования – псевдокласс **:hover**. Он используется для стилизации элементов при наведении на них указателя мыши.

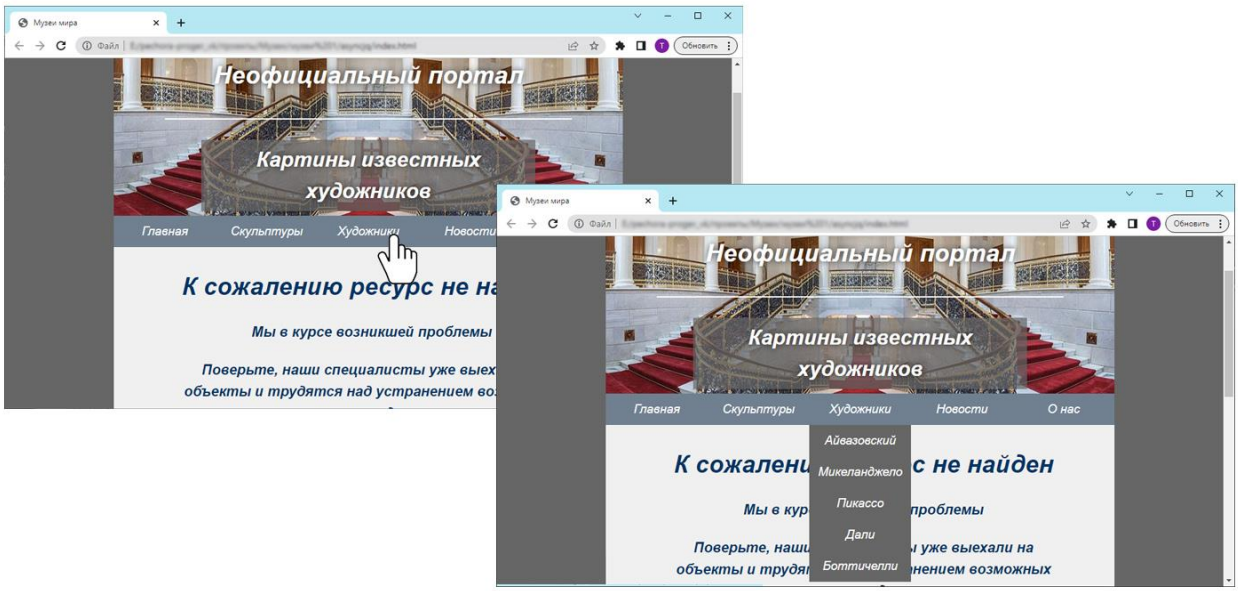
Таким образом, выделяем интерактивные элементы. Ссылки, на которые можно нажать, указатели выпадающих списков и т.д. В целом показываем пользователю, что элемент готов к взаимодействию. Чаще всего, так стилизуют ссылки, кнопки.



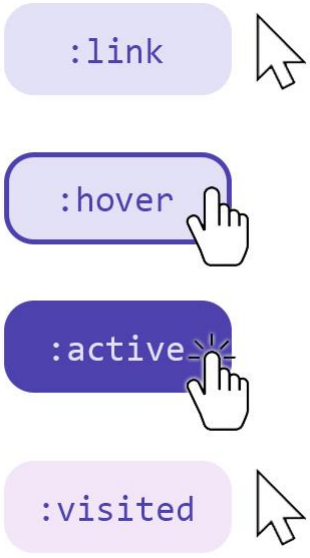
Имейте ввиду, что на мобильном или планшете **:hover** вряд ли сработает. А если и сработает, то только на некоторое время, после нажатия на элемент. Поэтому в своем проекте важно не иметь контент, **доступный только при наведении**, так как его будет неудобно или невозможно использовать на устройствах с сенсорными экранами.

©Pechora-PRO

С псевдоклассом **:hover** вы можете создавать сложные каскадные алгоритмы. Эта техника часто используется, например, чтобы создать выпадающие меню на «чистом» CSS без использования JavaScript. **Пример реализации в задании к лекции.**



:active



Псевдокласс **:active** соответствует элементу в момент, когда он активируется пользователем. Он позволяет странице среагировать, когда элемент активируется. При взаимодействии элемента с мышью - это, как правило, время между нажатием и отпусканием пользователем кнопки мыши.

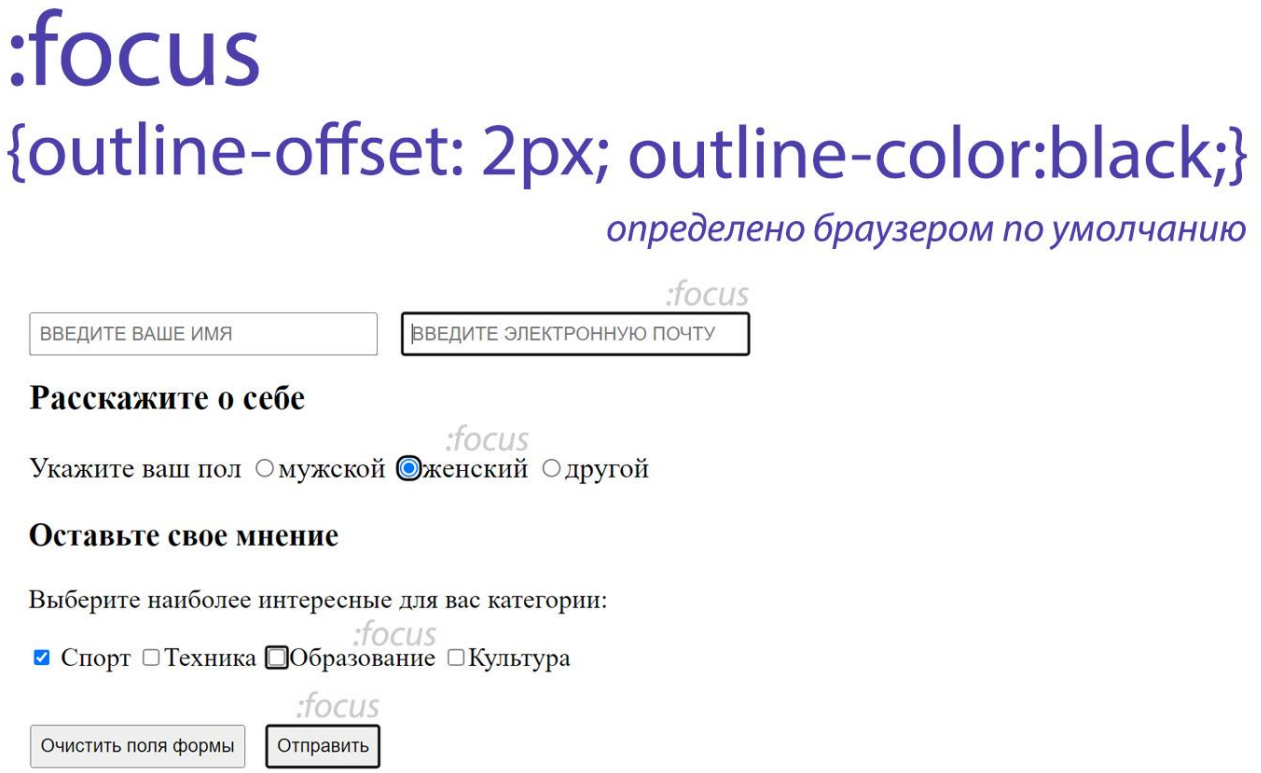
Псевдокласс **:active** обычно используется с элементами **a** и **button**, но может применяться и к другим элементам – например, элементам форм.

*Интересно, что в системах с много-кнопочными мышами, CSS 3 указывает, что псевдокласс **:active** должен применяться только по отношению к нажатию на первую кнопку; для праворуких мышей - это обычно самая левая кнопка.*

©Pechora-PRO

:focus

Еще один псевдокласс включается в работу, когда элемент (например, **input** формы) получает фокус. Обычно он активируется при клике мышью пользователем или при выборе элемента с использованием клавиши "tab" на клавиатуре.



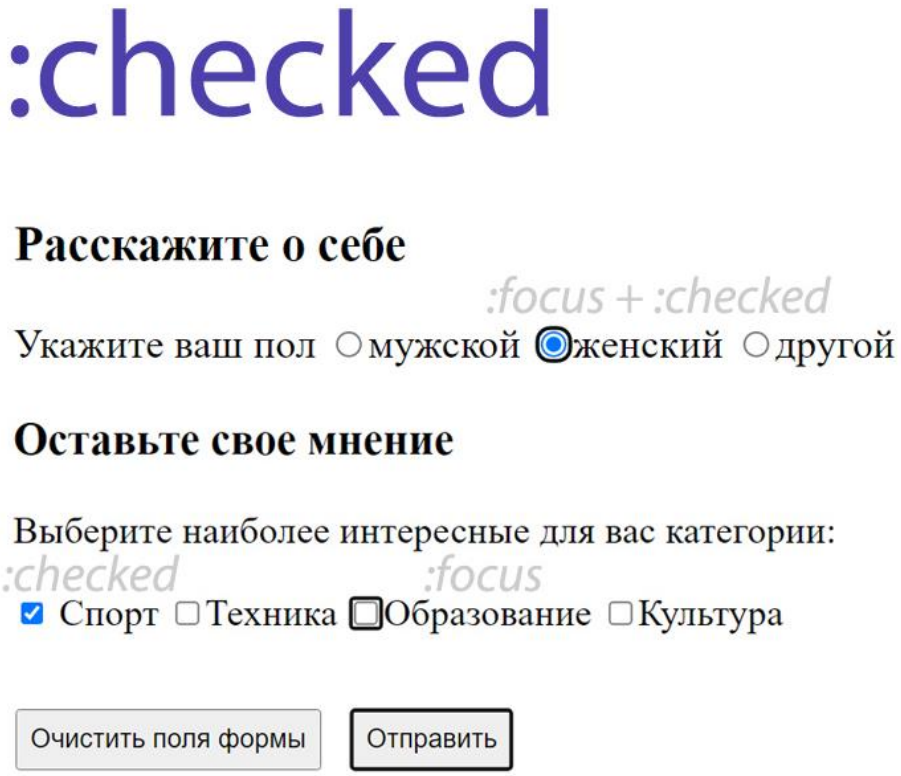
Для элементов «в фокусе» также дополнительно установлено свойство **outline** (точные настройки зависят от браузера). Не рекомендуется отключать его полностью, чтобы не нарушить доступность вашего элемента людям с ограниченными возможностями.

:checked

Раз уж мы заговорили об элементах формы, для многих из них важными **состояниями** являются: выбран/не выбран, включен/не включен.

CSS псевдокласс **:checked** находит любые элементы с типами radio, checkbox или option, которые выбраны или включены.

©Pechora-PRO



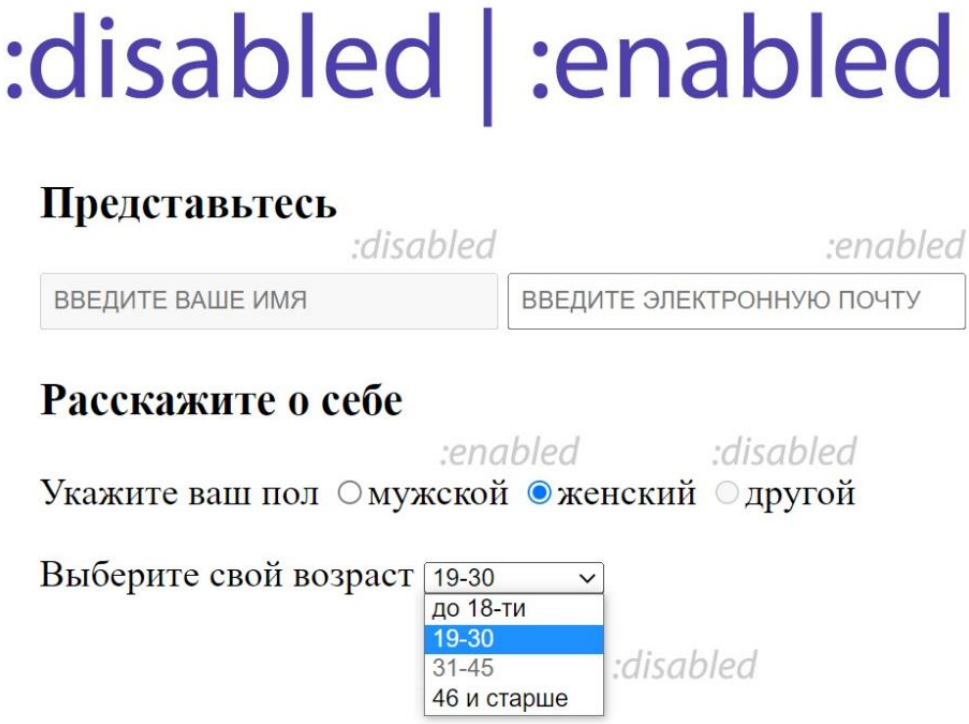
Пользователь может изменить это состояние, нажав на элемент, или выбрав другое значение, в этом случае **:checked** не будет повторно применен к элементу, а сохранит свое состояние.

:disabled, :enabled

Может случиться так, что нам понадобится настроить сразу все доступные для взаимодействия элементы, тогда сможем использовать псевдокласс **:enabled**.

Элемент включён, если его можно активировать (выбрать, нажать на него, ввести текст) или поставить фокус. У элемента также есть противоположное **отключённое** состояние, когда его нельзя активировать или сфокусировать. Тогда будем использовать псевдокласс **:disabled.**

©Pechora-PRO



:empty

Этот псевдокласс позволяет выбрать все элементы без потомков. В качестве содержимого будут учитываться любые символы, включая пробелы, кроме комментариев.

:valid, :invalid

Эти псевдоклассы позволят вам найти и выделить любые элементы **input** или **form**, контент которых валиден, в соответствии с типом поля. Они позволяют вам легко менять внешний вид полей, что помогает понять пользователю, ввёл он данные правильно или нет (см. Занятие 11. Валидация форм).

Если любая из радиокнопок в группе (т.е., с одинаковым атрибутом name) имеет атрибут **required** (обязательно для заполнения), псевдокласс **:invalid** применяется ко всем из них, если ни одна из кнопок группы не выбрана.

©Pechora-PRO

```
input:valid {  border: 2px solid green; color: green;}
input:invalid {  border: 2px solid red; color: red;}
```



Кстати, есть еще два псевдокласса для оформления элементов, связанные с обязательным заполнением.

CSS псевдокласс **:required** находит любые **input** элементы, имеющие атрибут **required**. В свою очередь, псевдокласс :**optional** может использоваться, чтобы дать особый внешний вид полям, не обязательным для заполнения.

:first-child, :first-of-type

:last-child, :last-of-type

С помощью этих значений будем искать элемент по положению в дереве, но не совсем обычно. Выделим на странице первый или последний элемент внутри одного родителя. А еще можно выделить все элементы – первые/последние вхождения тега: все первые/последние абзацы, списки, ячейки и т.д.

©Pechora-PRO

:first-of-type

```
div :first-of-type {
    background-color: rgb(20, 5, 108);
    color: white;
}
```

```
<div>
    <span>Это span первый!</span>
    <span>Этот span нет....</span>
    <span>А у этого есть <em>вложенный элемент</em>?</span>
    <p>Это вообще другой тег и он один, но ведь первый же...</p>
    <span>И такой уже был, значит тоже нет.</span>
</div>
```

Это span первый! Этот span нет.... А у этого есть *вложенный элемент*?

Это вообще другой тег и он один, но ведь первый же...

И такой уже был, значит тоже нет.

:nth-child, :nth-of-type

Выбрать первые и последние элементы хорошо, но иногда нужно выбрать второй… или третий… или, например, каждый четвертый.

Здесь нам поможет псевдокласс **:nth-child**, который находит один или более элементов, основываясь на их позиции среди группы соседних. Или **:nth-of-type**, который выберет указанные элементы нужного типа. В качестве аргумента можем передать разные значения:

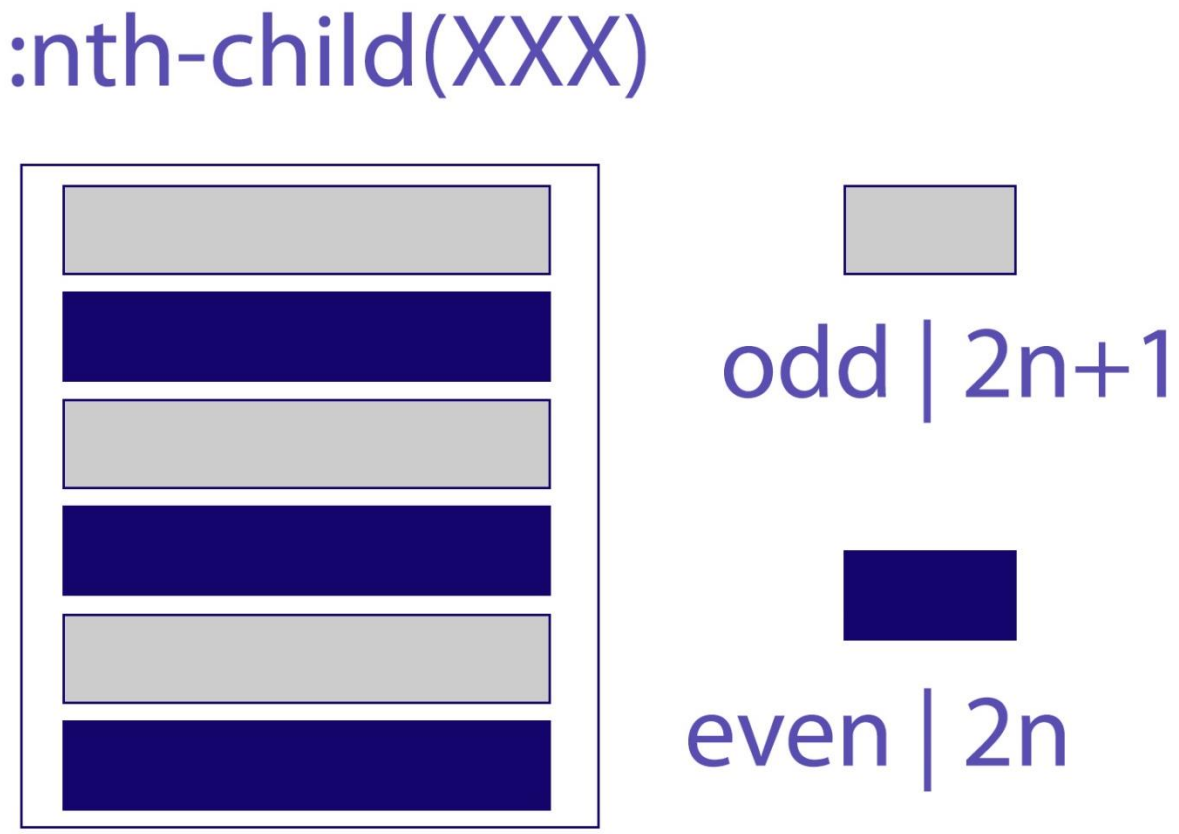
odd - описывает элементы среди группы соседних с нечётными номерами 1, 3, 5, и т. д.

even - описывает элементы среди группы соседних с чётными номерами 2, 4, 6, и т. д.

<An+B> - описывает элементы среди группы соседних, с номерами, соответствующими паттерну **An+B** (для каждого

©Pechora-PRO

целого числа n >= 0). Нумерация элементов начинается с единицы.



:not

Еще один псевдокласс **:not.**

Представляет элементы, которые **не соответствуют** списку селекторов. Поскольку он предотвращает выбор определенных элементов, он известен еще как псевдокласс отрицания.

:is, :where

Если при настройке страницы требуется задать одинаковое оформление для разных элементов, нужно использовать список селекторов.

Но, если какой-либо из них был недействителен, то весь список считается недействительным. При использовании **:is()** или **:where()** вместо того, чтобы весь список селекторов считался недействительным, если один из них не удалось проанализировать, неправильный или неподдерживаемый

©Pechora-PRO

селектор будет игнорироваться, а все остальные будут использоваться.

```
/*простой список селекторов*/
header>p, main>p {font-size: 2em;}

/*перечисление с :is*/
:is(header, main)>p  {font-size: 2em;}

/*перечисление с :where*/
:where(header, main)>p  {font-size: 2em;}
```

Псевдокласс :is может быть вложенным, например, выберем теперь только те параграфы, которые находятся в элементах div:

```
 :is(:is(header, main) div) > p  {font-size: 2em; }
```

Разница между псевдоклассами **:is** и **:where** в уровне специфичности.

Селекторы внутри **:where** имеют нулевую специфичность, поэтому настройки, заданные с его помощью могут быть переопределены простым селектором.

Псевдоэлементы

В отличие от псевдоклассов, псевдоэлементы, добавляемые к селектору, позволят стилизовать определённую часть выбранного элемента. Например, для изменения шрифта первой строки абзаца или первого символа строки или добавления нового контента. В селекторе можно использовать **только один псевдоэлемент**. Он должен находиться после простых селекторов в выражении.

Важно понимать, что, добавляя псевдоэлемент к селектору, мы как бы создаем новый элемент, который на странице отображается, но в DOM-дереве страницы не существует.

©Pechora-PRO

Такие несуществующие элементы могут помочь нам отредактировать текущий контент или создать принципиально новый.

Синтаксис практически идентичен псевдоклассам, разве что вместо одного двоеточия указываем два.

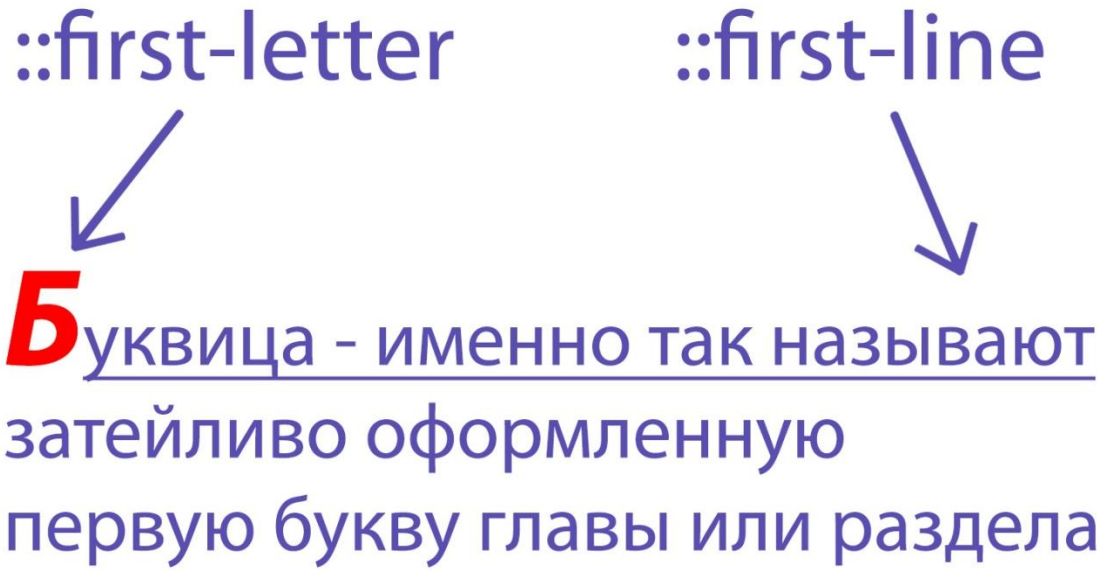
```
селектор::псевдокласс {свойство: значение;}
```

::first-line ::first-letter

Два псевдоэлемента выделяют первую строку и первый символ строки соответственно.

Важно чтобы первый символ действительно был первым, т.е. не имел перед собой другого содержимого (в том числе изображений и инлайновых таблиц).

Разработчики ограничили набор свойств, которые можно изменять с помощью этих псевдоэлементов. Те, которые доступны, связаны с настройками шрифта и текста, цветом, фоном, и границами, а также внешними и внутренними отступами (для ::first-letter).



::marker

©Pechora-PRO

В прошлой лекции говорил про настройку списков в целом и маркеров в частности. Мы задавали им отступы и меняли сами значки.

Псевдоэлемент **::marker** позволяет настраивать маркеру параметры так, как если бы это был отдельный самостоятельный элемент.

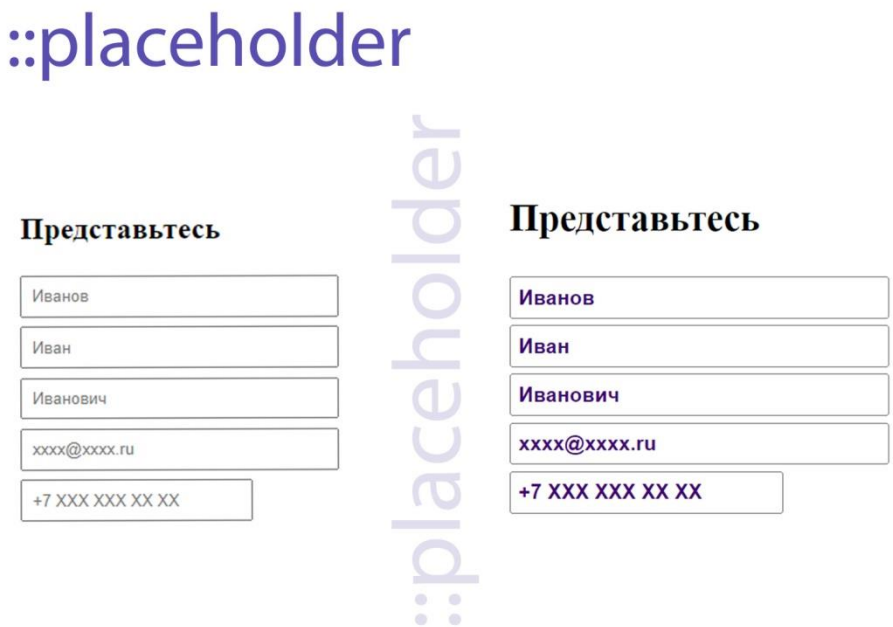
Допустима настройка любых параметров, которые касаются шрифта, свойства анимации и трансформации (рассмотрим их в следующей теме).

::placeholder

Еще один очень важный элемент страницы, который для нас был недоступен – **placeholder**-ы. Тот самый текст внутри поля ввода, который было бы очень здорово уметь стилизовать.

В большинстве браузеров по умолчанию **placeholder** выглядит как полупрозрачный или светло-серый текст.

Но, благодаря псевдоэлементу **::placeholder**, это можно изменить. Список разрешенных свойств такой же как и для **::first-line**.



::selection

©Pechora-PRO

Псевдоэлемент **::selection** позволяет применить стили к части документа, который был выделен пользователем (например, с помощью мыши). Это позволит вам стилизовать свой документ не только в статичном состоянии, но и при взаимодействии с пользователем.

Список разрешенных свойств также ограничен: цвет текста и фон (кроме background-image), тень текста, обводка, группа свойств text-decoration и др.

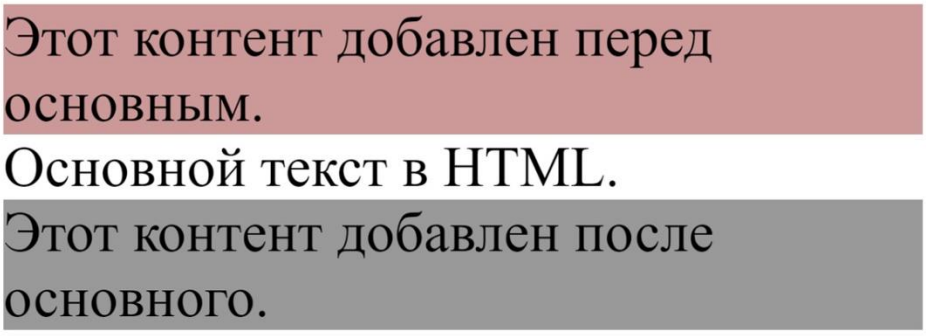
::before, ::after

::before и **::after** создают псевдоэлементы, которые являются первым и последним потомком выбранного элемента соответственно. Часто используется для добавления дополнительного содержимого в элемент с помощью свойства **content**.

```html
<html>
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Псевдоклассы и псевдоэлементы</title>
        <link rel="stylesheet" type="text/css" href="style.css">
    </head>
    <body>
        <p>Основной текст в HTML.</p>
    </body>
```

```css
p::before {
  content: "Этот контент добавлен перед основным. ";
  background-color: #cc9999;
  float: left;
}

p::after {
  content: "Этот контент добавлен после основного. ";
  background-color: #999999;
  float: left;
}
```



©Pechora-PRO

CSS-свойство **content** заменяет элемент сгенерированным значением. Объекты, добавленные с помощью свойства **content,** являются анонимными замещаемыми элементами. Анонимные, потому что не существуют в html-разметке. Замещаемые, потому что их представление не зависит от модели форматирования css.

Важно! Достаточно большое количество псевдоклассов и псевдоэлементов не поддерживаются одним или несколькими браузерами. Будьте внимательны при их использовании, если в вашем проекте важно обеспечить кроссбраузерную поддержку.

©Pechora-PRO